

Ballistics System

물리 기반 탄도 시스템 | UE5 C++ | CoD Clone Project

UActorComponent 직접 상속 · SweepSingleByChannel · 오일러 적분

v1.0

왜 직접 구현인가

Engine Default

// 컴포넌트가 알아서 처리

```
ProjectileMovement->InitialSpeed = 3000.0f;  
ProjectileMovement->MaxSpeed = 3000.0f;  
ProjectileMovement->bShouldBounce = true;
```

- ✗ 내부 동작 불투명
- ✗ 파라미터 범위 내에서만 제어 가능
- ✗ 물리 모델 자체 수정 불가



UCOD_PMC

- ✓ 탄도 공식을 직접 코드로
- ✓ 충돌 판정 직접 호출
- ✓ 엔진 컴포넌트 미상속
- ✓ UActorComponent 직접 출발

```
// UProjectileMovementComponent 교체  
ProjectileMovement =  
    CreateDefaultSubobject<  
        UCOD_ProjectileMovementComponent>(TEXT(...));
```

SetActorLocation의 한계 — 왜 Sweep인가

SetActorLocation — 텔레포트 방식



SweepSingleByChannel — 경로 전체 스캔



경로 위 모든 충돌체를 구형($r=5\text{cm}$) 스캔으로 검사 → 고속 발사체도 관통 없음

탄도 물리 구현 — 오일러 적분

탄도식 (Prototype v1)

$$V(t+dt) = V(t) + A \times dt$$

$$A = \{ 0, 0, -GravityScale \times 980 \}$$

(단위: cm/s^2)

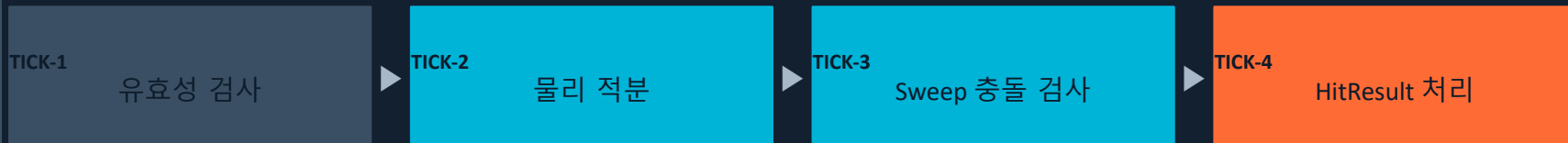
$$\Delta = V(t) \times dt$$

ApplyGravity()

```
FVector UCOD_PMC::ApplyGravity()
{
    FVector Accel = {
        0.f, 0.f,
        -GravityScale * 980.f };
    return Accel;
}
```

GravityScale로 직접 배율 관리 → 월드 설정에 묶이지 않음
발사체별 독립 중력 파라미터 가능
1 UU $\approx$ 1 cm / 초기속도 2000 = 초속 20m

TickComponent — 매 프레임 흐름



충돌 O → OnHitDelegate.Broadcast | 충돌 X → SetWorldLocation + SetWorldRotation

충돌 판정 + 델리게이트 패턴

SweepSingleByChannel

Chaos와의 직접 통신 — 최저 레벨 공개 API

```
FHitResult HitResult;
bool bHit =
    GetWorld()->

    SweepSingleByChannel(
        HitResult,

        CurrentPos, // Start

        NewPos,      // End
        FQuat::Identity,
        ECC_Visibility,
        MakeSphere(
            5.f), // r=5cm
        Params
    );
```

MoveComponent() 내부에서도 이것을 호출
이보다 아래는 Chaos 블랙박스

델리게이트 기반 충돌 처리

```
DECLARE_DYNAMIC_MULTICAST_DELEGATE
    _OneParam(

    FOnHitDelegate,
        const FHitResult&, Hit
    );
```

UCOD_PMC
충돌 감지

OnHitDelegate
.Broadcast()

ShooterProjectile
OnProjectileHit()

컴포넌트는 충돌 후 행동을 모른다

→ 역할 분리, 재사용성 확보

→ NotifyHit 대신 커스텀 핸들러 사용

회전 동기화 — CalcRotation()

CrossProduct 기반 직교 행렬

```
FVector Forward = _Velocity.GetSafeNormal();  
FVector Up      = FVector::UpVector;  
FVector Right   = FVector::CrossProduct  
    CrossProduct(Up, Forward).  
    GetSafeNormal();  
Up = FVector::CrossProduct  
    (Forward, Right).GetSafeNormal();  
  
FMatrix m(Forward, Right, Up, Zero);  
return m.Rotator();
```

매 Tick 속도 벡터 방향으로 메시 회전 맞춤

Forward → Right → Up 재직교화 → FMatrix → FRotator

Up을 재계산하는 이유: 월드 UpVector와 Forward가 완전히 수직이

아닐 수 있어 FMatrix 직교성 보장 필요

⚠ 짐벌락 발생 조건 인지

```
Forward  = (0,0,-1) ← 수직 낙하  
UpVector = (0,0, 1)  
CrossProduct → (0,0,0) ← 영벡터  
→ 다음 단계: FQuat 교체 예정
```

핵심 어필 포인트

● 오일러 적분 직접 구현

● Sweep 관통 방지

● Chaos 최저 레벨 API

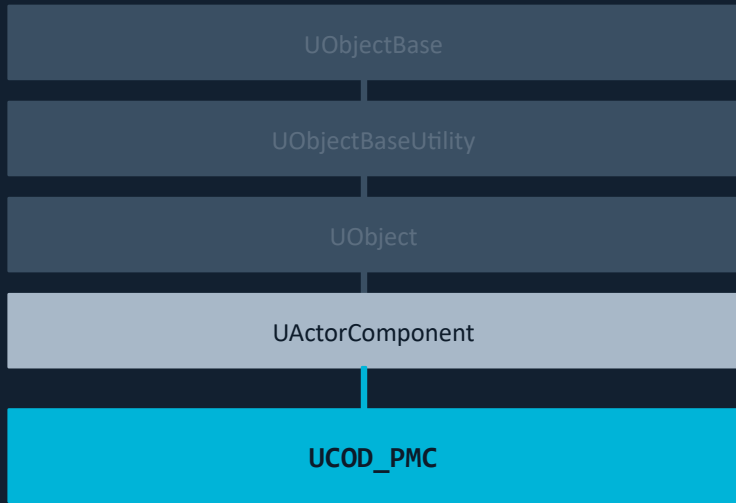
● UActorComponent 직접 상속

● 델리게이트 역할 분리

● 짐벌락 구조적 인지

클래스 구조 + 현재 상태

상속 구조



직접 처리하는 것들:

- `UpdatedComponent` → `BeginPlay` 직접 바인딩
- 위치 갱신 → `SetWorldLocation` 직접 호출
- 충돌 판정 → `SweepSingleByChannel` 직접 호출

현재 상태

✓	탄도 물리 (중력)	오일러 적분, <code>GravityScale</code>
✓	Sweep 충돌 판정	<code>SweepSingleByChannel</code> , <code>r=5cm</code>
✓	회전 동기화	CrossProduct 기반 <code>FMatrix</code>
✓	디버그 시각화	<code>DrawDebugLine</code> 궤적 확인
✓	델리게이트 연동	<code>OnHitDelegate</code> → <code>ShooterProjectile</code>
○	AirDrag 공기저항	헤더 선언 완료, 적분 대기
○	짐벌락 해소	<code>FQuat</code> 교체 예정
○	오브젝트 풀링	<code>SpawnActor/Destroy</code> 현행